Personal Project

Pipelined Processor
5 stages

RV32I

Verilog

Yacine Ouhrouche

Started:

16/03/26

Table of Contents

1. Project Overview 3

2 . Development Environment 3

3 . CPU Architecture 4

4. RV32I Instruction Format 4

5. Control Signals 8

6. Hardware Modules 10

7. Hazard Detection 13

8. Forwarding 13

9.Datapath 14

10 . Instruction Pipeline Example 16

11.RTL Files Organization 17

12. Verification 18

13 .Design Analysis 26

14 .Performance 29

15. Power Analysis 30

16.Limitation Of the Design 31

17 .Summary 31

18 .Future Work 31

1. Project Overview

This project consists of the design and implementation of a 32-bit 5-stage pipelined RISC-V processor supporting a subset of the RV32I base integer instruction set**.** The processor is implemented in Verilog HDL and follows a Harvard memory architecture**,** separating instruction and data memory to avoid structural hazards. This design builds upon a previously developed single-cycle RISC-V processor, extending the architecture to a pipelined execution model to improve instruction throughput and overall performance.

The processor implements the standard five pipeline stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB)**.** A hardwired control unit is used to generate control signals efficiently while maintaining a modular and manageable design consistent with the RISC-V specification.

2. Development Environment

The processor was developed and verified using:

- **VSCode**: Used for writing and managing the Verilog and SystemVerilog source files.
- **Verilog**: Used to implement the RTL design.
- **SystemVerilog** : Used to develop testbenches for functional verification.
- **SystemVerilog Assertions (SVA)**: Used within the testbench to check important design properties and detect incorrect behavior during simulation.
- **Verilator (Lint Mode)**: Detect syntax issues, unused signals, and potential design problems before simulation.
- **Icarus Verilog**: Used for compiling and simulating the Verilog/SystemVerilog design.
- **Surfer** :Used to visualize simulation waveforms.
- **Yosys**: Used to synthesize RTL design and estimate hardware resource usage.
- **OpenSTA**: Used to perform STA on the synthesize design and estimate critical path and potential clock frequency.

3. CPU Architecture Specifications

- Architecture: RV32I
- Data Width: 32-bit
- Instruction Width: 32-bit
- Register Count: 32 (x0–x31)
- Memory Architecture: Harvard
- Execution Model: 5-stage pipelined
- Stages : Instruction Fetch (IF), Instruction Decode(ID), Execute(EX), Memory Access(MEM), Write Back(WB)

4. RV32I Instruction Format

4.1 R-Type Format

31	25	24	20	19	15	14	12	11	7	6	0
Func 7		Rs2		Rs1		Func3		rd		Opcode	

R-Type Instructions

Name	Func7	Func3	Descr	Opcode
ADD	0000000	000	rd = rs1 + rs2	0110011
SUB	0100000	000	rd = rs1 - rs2	0110011
XOR	0000000	100	rd = rs1 ^ rs2	0110011
OR	0000000	110	rd = rs1 | rs2	0110011
AND	0000000	111	rd = rs1 & rs2	0110011
SLL	0000000	001	rd = rs1 << rs2	0110011
SRL	0000000	101	rd = rs1 >> rs2	0110011
SRA	0100000	101	rd = rs1 >> rs2	0110011
SLT	0000000	010	d = (rs1 < rs2) ? 1 : 0	0110011
SLTU	0000000	011	rd = (rs1 < rs2) ? 1 : 0	0110011

4.2 I-Type Format

31	20	19	15	14	12	11	7	6	0
imm		Rs1		Func3		rd		Opcode	

I-Type Instructions

Name	Func7	Func3	Descr	Opcode
ADDI	-	000	rd = rs1 + imm	0010011
XORI	-	100	rd = rs1 ^ imm	0010011
ORI	-	110	rd = rs1 | imm	0010011
ANDI	-	111	rd = rs1 & imm	0010011
SLLI	0000000	001	rd = rs1 << imm	0010011
SRLI	0000000	101	rd = rs1 >> imm	0010011
SRAI	0100000	101	rd = rs1 >> imm	0010011
SLTI	-	010	d = (rs1 < imm) ? 1 : 0	0010011
SLTUI	-	011	rd = (rs1 < imm) ? 1 : 0	0010011

I-Type Loads

Name	Func7	Func3	Descr	Opcode
LB	-	000	rd = M[rs1+imm][0:7]	0000011
LH	-	001	rd = M[rs1+imm][0:15]	0000011
LW	-	010	rd = M[rs1+imm][0:31]	0000011
LBU	-	100	rd = M[rs1+imm][0:7]	0000011
LHU	-	101	rd = M[rs1+imm][0:15]	0000011

4.3 S-Type Format

31	20	19	15	14	12	11	7	6	0
Imm[11:5]		Rs1		Func3		Imm[4:0]		Opcode	

S-Type Instructions

Name	Func7	Func3	Descr	Opcode
SB	-	000	M[rs1+imm][0:7] = rs2[0:7]	0100011
SH	-	001	M[rs1+imm][0:15] = rs2[0:15]	0100011
SW	-	010	M[rs1+imm][0:31] = rs2[0:31]	0100011

4.4 B-Type Format

31	25	24	20	19	15	14	12	11	7	6	0
Imm[12|10:5]		Rs2		Rs1		Func3		Imm[4:1|11]		Opcode	

B type Instruction

Name	Func7	Func3	Descr	Opcode
BEQ	-	000	If(rs1==rs2) pc+=im	1100011
BNE	-	001	If(rs1!=rs2) pc+=im	1100011
BLT	-	100	If(rs1<rs2) pc+=im	1100011
BGE	-	101	If(rs1>=rs2) pc+=im	1100011
BLTU	-	110	If(rs1<rs2) pc+=im	1100011
BGEU	-	111	If(rs1>=rs2) pc+=im	1100011

4.5 U-Type Format

31	12	11	7	6	0
Imm[31:12]		rd		Opcode	

U-Type Instructions

Name	Func7	Func3	Descr	Opcode
LUI	-	-	Rd = imm << 12	0110111
AUIPC	-	-	Rd = pc+(imm<<12)	0010111

4.6 J-Type Format

31	12	11	7	6	0
Imm[20|10:1|11|19:2]		rd		Opcode	

J-Type Instructions

Name	Func7	Func3	Descr	Opcode
JAL	-	-	Rd = pc+4 pc+= imm	1101111
JALR	-	000	Rd=pc+4, pc = rs1 + imm	1100111

5. Control Signal

5.1 Overview

The control logic decodes the instruction fields (opcode, funct3, and funct7) in the Instruction Decode (ID) stage and generates control signals that determine how data flows through the datapath. These signals control multiplexers, enable register writes, select ALU operations, control data memory access, and determine the next value of the program counter.

In the pipelined design, the generated control signals are stored in pipeline registers (ID/EX, EX/MEM, MEM/WB) and propagated to the appropriate pipeline stages where they are used. This ensures that each instruction carries its corresponding control signals as it moves through the pipeline.

For branch instructions, the Branch Comparator produces the signals BrEq and BrLT, which are used together with the instruction's funct3 field and the BrUn signal to determine whether a branch condition is satisfied.

5.2 List of Signals

Signal	**Width**	**Description**
PCSel	1	Select next PC
ImmSel	3	Immediate formats select
REgWEn	1	Register writes enable
BrUn	1	Branch compare
ASel	1	ALU input A select
BSel	1	ALU input B select
ALUSel	4	ALU operation select
MemRW	1	Data memory direction
WBSel	2	Writeback select

5.2 Control Signals by Stages

Stages	Signals
EX	ASel, BSel, ALUSel, PCSel,
MEM	MemRW
WB	RegWEn, WBSel

5.4 Control signal by Instruction Type

Legend:

- 0= disabled
- 1 = enabled
- X = don't care

Type	PCSel	ImmSel	RegWEn	BrUn	ASel	BSel	ALUSel	MemRW	WBSel
R	0	xxx	1	X	0	0	decode	0	01
I	0	000	1	X	0	1	decode	0	01
Load	0	000	1	X	0	1	add	0	00
S	0	001	0	X	0	1	add	1	xx
B	depend	010	0	depend	1	1	add	0	xx
JAL	1	100	1	x	1	1	add	0	10
JALR	1	000	1	X	0	1	add	0	10
LUI	0	011	1	X	X	1	Pass_b	0	01
AUIPC	0	011	1	x	1	1	add	0	01

6. Hardware Modules

IF Stage

6.1 Program counter

The Program Counter (PC) stores the address of the current instruction being executed. At each clock edge, the PC is updated with the next instruction address selected by the PC multiplexer. Normally, the PC is incremented by 4 to fetch the next sequential instruction, but it may also be updated with a branch or jump target address.

6.2 Instruction Memory

The instruction memory stores the program instructions executed by the processor. It is accessed using the address provided by the PC. At each cycle, the instruction located at the address PC is fetched and provided to the datapath for decoding and execution.

In this implementation, instruction memory is read-only during execution and follows the Harvard architecture, meaning that instructions and data are stored in separate memory modules. This avoids structural hazards and allows instruction fetch and data access to occur independently.

6.3 Adder

The adder is used to compute the next sequential instruction address. In this implementation, a single adder calculates PC + 4, which is used to fetch the next instruction in program order.

ID Stage

6.4 Register file

The register file contains 32 general-purpose registers used for operand storage. Two registers can be read simultaneously through the rs1 and rs2 ports, while a third register may be written through the rd port when RegWEn is enabled. Register x0 is hardwired to zero and cannot be modified.

6.5 Immediate Generator

The Immediate Generator extracts and sign-extends immediate values from the instruction word according to the instruction format. The specific immediate format is selected using the ImmSel control signal.

6.6 Control Unit

The Control Unit decodes the instruction opcode and function fields to generate control signals that steer the datapath. These signals control multiplexers, register writes, ALU operations, memory accesses, and the next program counter selection.

6.7 Branch Comparator

The Branch Comparator evaluates the relationship between rs1 and rs2 to determine branch conditions. It generates the BrEq and BrLT signals used by the control logic to determine whether a branch should be taken.

EX Stage

6.8 ALU

The Arithmetic Logic Unit performs arithmetic, logical, and shift operations required by the instruction set. The operation executed by the ALU is determined by the ALUSel control signal. The operands are selected using the ASel and BSel multiplexers.

6.9 Multiplexers

Multiplexers are used throughout the datapath to select between multiple data sources. They control the selection of ALU operands, writeback data, and the next program counter value.

MEM Stage

6.10 Data Memory

The data memory is used for load and store instructions. The memory address is provided by the ALU result, which represents the effective address computed from a base register and an immediate offset.

Pipeline Support

6.11 Pipeline Registers

Pipeline registers are placed between each pipeline stage to store intermediate data and control signals as instructions progress through the pipeline. These registers isolate the pipeline stages and allow multiple instructions to execute simultaneously.

The processor uses four pipeline registers:

- **IF/ID**: stores the fetched instruction and PC+4
- **ID/EX**: stores register operands, immediate values, and control signals
- **EX/MEM**:stores ALU results and memory control signals
- **MEM/WB**: stores memory outputs and writeback control signals

6.12 Forwarding Unit

The Forwarding Unit resolves data hazards by forwarding results from later pipeline stages to earlier stages. Instead of waiting for results to be written back to the register file, the forwarding unit routes the values directly from the EX/MEM or MEM/WB pipeline registers to the ALU inputs when necessary.

6.13 Hazard Detection Unit

The Hazard Detection Unit monitors instructions in the pipeline to detect situations where data dependencies cannot be resolved through forwarding alone. When a load-use hazard occurs, the unit temporarily stalls the pipeline by preventing updates to the PC and IF/ID register while inserting a bubble into the pipeline.

7. Hazard Detection

Hazards can occur in a pipelined processor when instructions depend on results that are not yet available. These situations are classified as data hazards, control hazards, and structural hazards. Structural hazards are avoided in this design by using separate instruction and data memories.

Data hazards are reduced using data forwarding, which allows results from later pipeline stages to be sent directly to the execute stage. However, load-use hazards cannot be resolved through forwarding because the data becomes available only after the memory stage. In this case, the hazard detection unit inserts a stall cycle.

Control hazards occur when branch instructions change the program flow. If a branch is taken, incorrectly fetched instructions are flushed and the program counter is updated with the correct target address.

8. Forwarding

In a pipelined processor, data hazards may occur when an instruction requires the result of a previous instruction that has not yet been written back to the register file. Without additional mechanisms, this would require the pipeline to stall until the value becomes available.

To reduce unnecessary stalls, the processor implements data forwarding (also known as bypassing). Forwarding allows results produced in later pipeline stages to be routed directly to the execute stage where they are needed. Instead of waiting for the value to be written back to the register file, the value is forwarded from the EX/MEM or MEM/WB pipeline registers to the ALU inputs.

This mechanism resolves most data hazards and allows dependent instructions to execute without introducing pipeline stalls. However, forwarding cannot resolve load-use hazards, since the data from a load instruction becomes available only after the memory stage. In these cases, the hazard detection unit inserts a stall cycle to ensure correct execution.

By forwarding results between pipeline stages, the processor significantly reduces the number of stalls and improves overall pipeline efficiency.

9. Datapath

The datapath defines how data moves through the processor during instruction execution. The processor implements a five-stage pipelined architecture consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Pipeline registers separate each stage, allowing multiple instructions to be processed simultaneously.
The overall datapath structure is shown in the datapath diagram below, which illustrates the main hardware components and the flow of data between pipeline stages.

IF
In the fetch stage, the program counter (PC) provides the address of the instruction to the instruction memory. The fetched instruction and the value PC + 4 are stored in the IF/ID pipeline register. A multiplexer controlled by PCSel selects the next value of the program counter, allowing either sequential execution or a branch/jump target.

ID
During the decode stage, the instruction fields are extracted and interpreted. The register file reads operands from registers rs1 and rs2, while the immediate generator produces the appropriate immediate value. The control unit generates the control signals required for instruction execution. The decoded operands, immediate value, and control signals are stored in the ID/EX pipeline register.

EX
In the execute stage, the ALU performs arithmetic, logical, and shift operations. Multiplexers select the ALU operands from either register values, the program counter, or the generated immediate. Branch instructions are evaluated using the branch comparator. The ALU result and associated signals are stored in the EX/MEM pipeline register.

MEM
In the memory stage, the processor accesses data memory for load and store instructions. The ALU result provides the memory address used for the operation. Load instructions read data from memory, while store instructions write register data to memory. The results are stored in the MEM/WB pipeline register.

WB
In the write-back stage, the result of the instruction is written to the register file. A multiplexer controlled by WBSel selects between the ALU result, memory data, or PC + 4 for jump instructions.

Once the pipeline is filled, the processor can ideally complete one instruction per clock cycle, improving throughput compared to a single-cycle design.

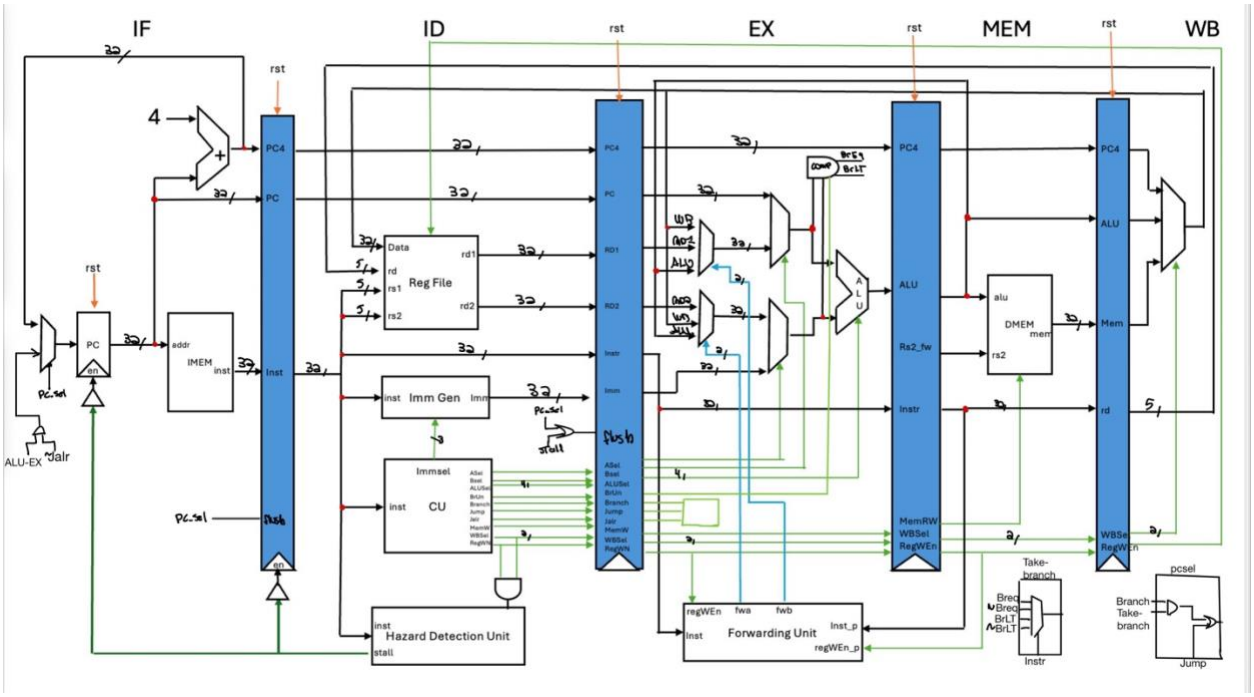

Figure 1: Datapath Representation

10. Pipeline Timing Diagram

10.1 Normal Pipeline Execution

Cycle	1	2	3	4	5	6	7
addi x1,x0,5	IF	ID	EX	MEM	WB		
addi x2, x0,10		IF	ID	EX	MEM	WB	
add x3, x1, x2			IF	ID	EX	MEM	WB
sub x4, x3,x2				IF	ID	EX	MEM

10.2 Pipeline with Data Hazard (Forwarding)

Cycle	1	2	3	4	5	6	7
add x1,x2,x3	IF	ID	EX	MEM	WB		
add x4,x1,x5		IF	ID	EX	MEM	WB	

10.3 Load-Use Hazard (Stalling)

Cycle	1	2	3	4	5	6	7
lw x1, 0(x2)	IF	ID	EX	MEM	WB		
add x3,x1,x4		IF	ID	EX	MEM	WB	

Stalled:

Cycle	1	2	3	4	5	6	7
lw x1, 0(x2)	IF	ID	EX	MEM	WB		
Nop(bubble)		IF	ID	EX	MEM	WB	
add x3,x1,x4			IF	ID	EX	MEM	WB

10.4 Pipeline with Control Hazard (Flush)

Cycle	1	2	3	4	5	6	7
beq x1,x2,L	IF	ID	EX	MEM	WB		
add x3,x4,x5		IF	ID	EX	MEM	WB	
sub x6, x7,x8			IF	ID	EX	MEM	WB

Flush:

Cycle	1	2	3	4	5	6	7
beq x1,x2,L		ID	EX	MEM	WB		

11.RTL File Organization

The processor is implemented using a modular RTL design written in Verilog. The project is organized into multiple directories separating the core processor logic, hardware blocks, memory modules, and verification files.

The folder structure of the project is shown below:

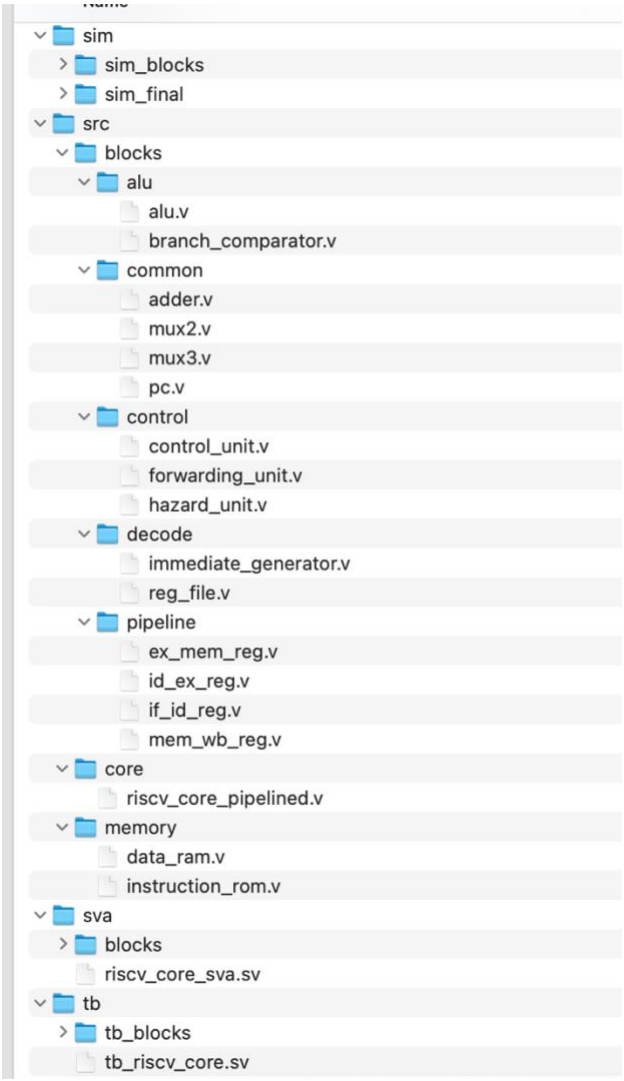

Figure 2: RTL File organization

12. Verification

The processor was verified using modular testbenches. Each hardware block was first tested independently using directed tests and SystemVerilog assertions. After validating individual modules, they were integrated and tested in the full CPU simulation to confirm correct pipeline behavior.

12.1 ALU

The ALU was verified by varying operands a and b while changing ALUSel. Operations including ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, and SLTU. The waveform confirms correct result selection and proper zero flag behavior.

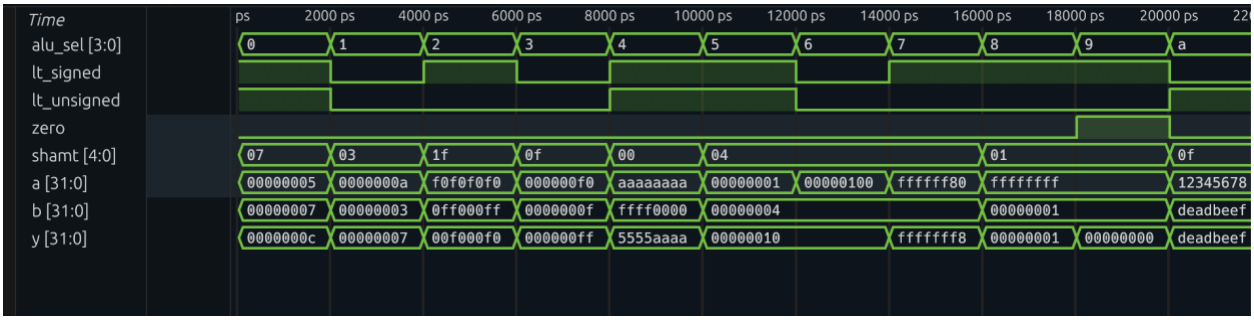

Figure 3: ALU Simulation Waveform

12.2 Register File

The register file was verified by performing writes on the rising clock edge and reading values through rs1 and rs2. The waveform shows correct synchronous writes and simultaneous reads.

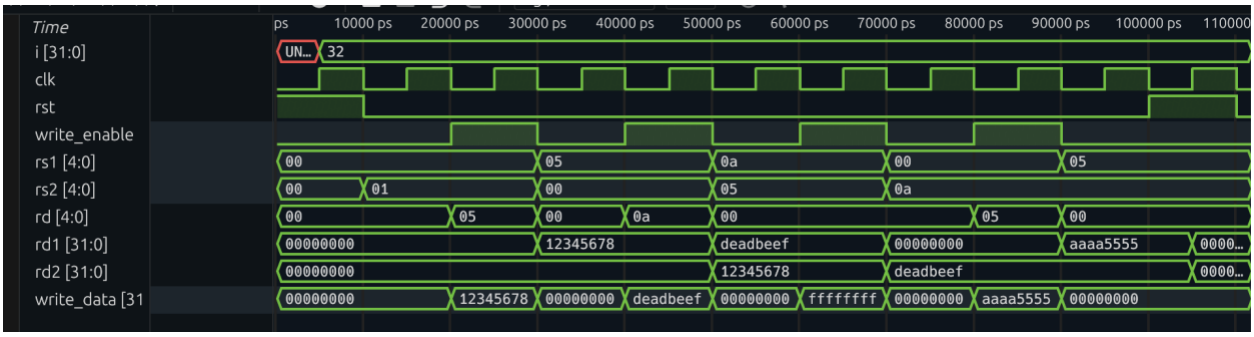

Figure 4: Reg File Simulation Waveform

12.3 Immediate Generator

Instructions from the I, S, B, U, and J formats were applied to verify immediate extraction. The generated immediate values match the expected results, confirming correct bit extraction and sign extension.

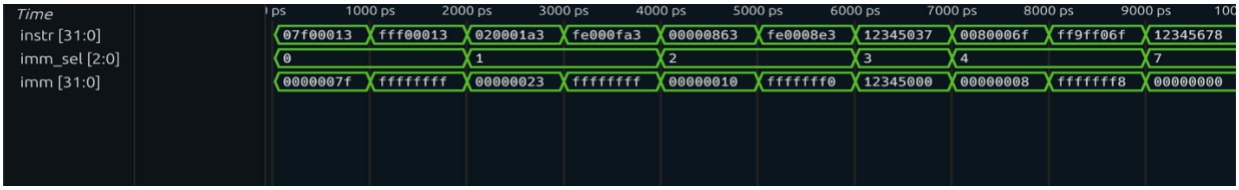

Figure 5: Immediate Gen Simulation Waveform

12.4 Control Unit

The control unit was verified using a directed testbench that applied instructions from each instruction class. Tests included ADD, SUB, AND, ADDI, SRAI, LW, SW, BEQ, BLTU, JAL, JALR, LUI, AUIPC, and a default case. All tests passed, confirming correct instruction decoding and control signal generation.

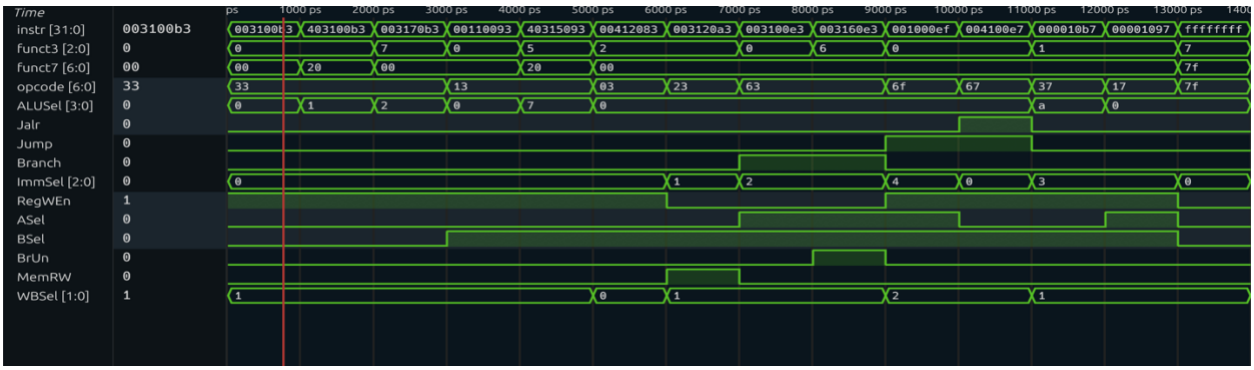

Figure 6: Control Unit Simulation Waveform

12.5 Branch Comparator

The branch comparator was verified by applying different operand values and observing the comparison outputs. The simulation confirms correct signed and unsigned comparisons used for branch instructions.

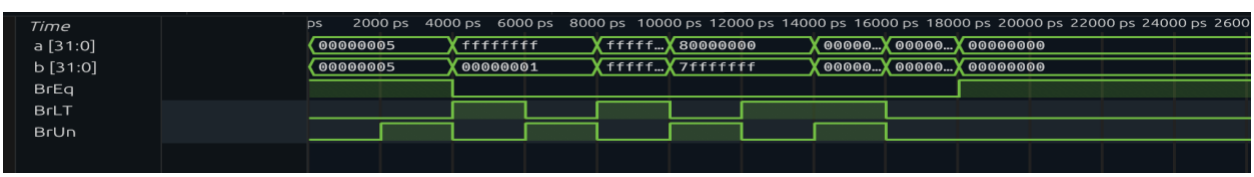

Figure 7: Branch Comparator Simulation Waveform

12.6 Program Counter

The program counter was verified by applying a reset and observing updates on each clock cycle. The waveform confirms correct initialization and sequential updates where PC = PC + 4.

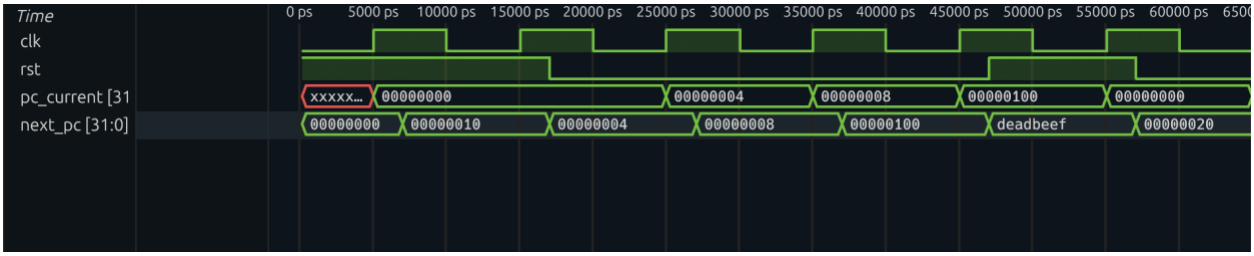

Figure 8: PC Simulation Waveform

12.7 Pipeline Register (IF_ID)

Pipeline registers were verified by applying program counter and instruction inputs while varying rst, en, and flush control signals. The waveform confirms correct propagation of values on clock edges, proper stalling when en is disabled, and insertion of a NOP when flush is asserted. The IF/ID register is shown as an example, since all pipeline registers share the same synchronous register structure and behavior

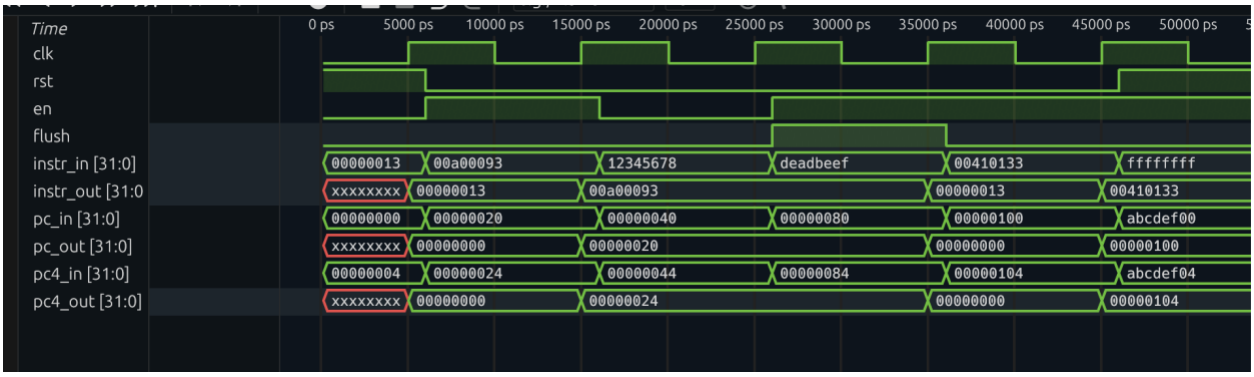

Figure 9: IF_ID Register Simulation Waveform

12.8 Hazard Detection Unit

The hazard detection unit was verified by applying different load and register dependency conditions and observing the `stall` output. The simulation confirms that a stall is asserted only when the instruction in the ID/EX stage is a load and its destination register matches one of the source registers of the instruction in the IF/ID stage. This ensures correct detection of load-use hazards in the pipeline.

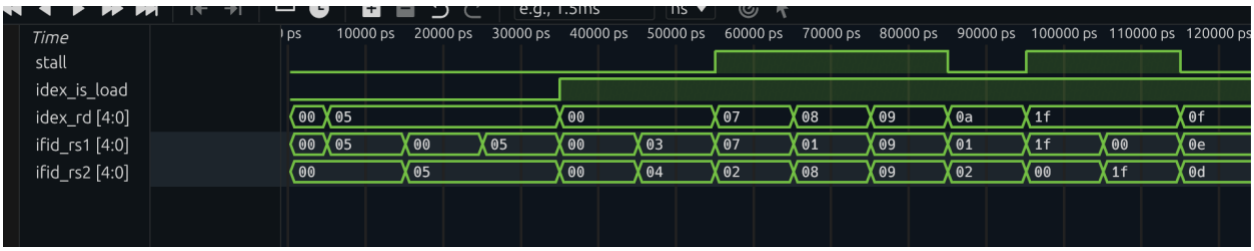

Figure 10: Hazard Detection Unit Simulation Waveform

12.9 Forwarding Unit

The forwarding unit was verified by applying different register write and dependency conditions and observing the ForwardA and ForwardB outputs. The simulation confirms that forwarding occurs when a source register in the ID/EX stage matches the destination register of an instruction in the EX/MEM or MEM/WB stage, with EX/MEM having priority. This ensures correct data forwarding in the pipeline.

Figure 11: Forwarding Unit Simulation Waveform

12.10 CPU

The CPU was verified using four programs. Key signals include PC, instruction, pipeline registers, ALU results, and control signals (stall, flush, ForwardA,ForwardB).

12.10.1 Normal code

This test verifies basic execution without hazards. The pipeline operates normally with stall = 0 and flush = 0, and the PC increments sequentially. Instructions flow correctly through all stages.

```
00500093   # addi x1, x0, 5
00A00113   # addi x2, x0, 10
01400193   # addi x3, x0, 20
40208233   # sub  x4, x1, x2
0021E2B3   # or   x5, x3, x2
00117333   # and  x6, x2, x1
00402023   # sw   x4, 0(x0)
00002383   # lw   x7, 0(x0)
```

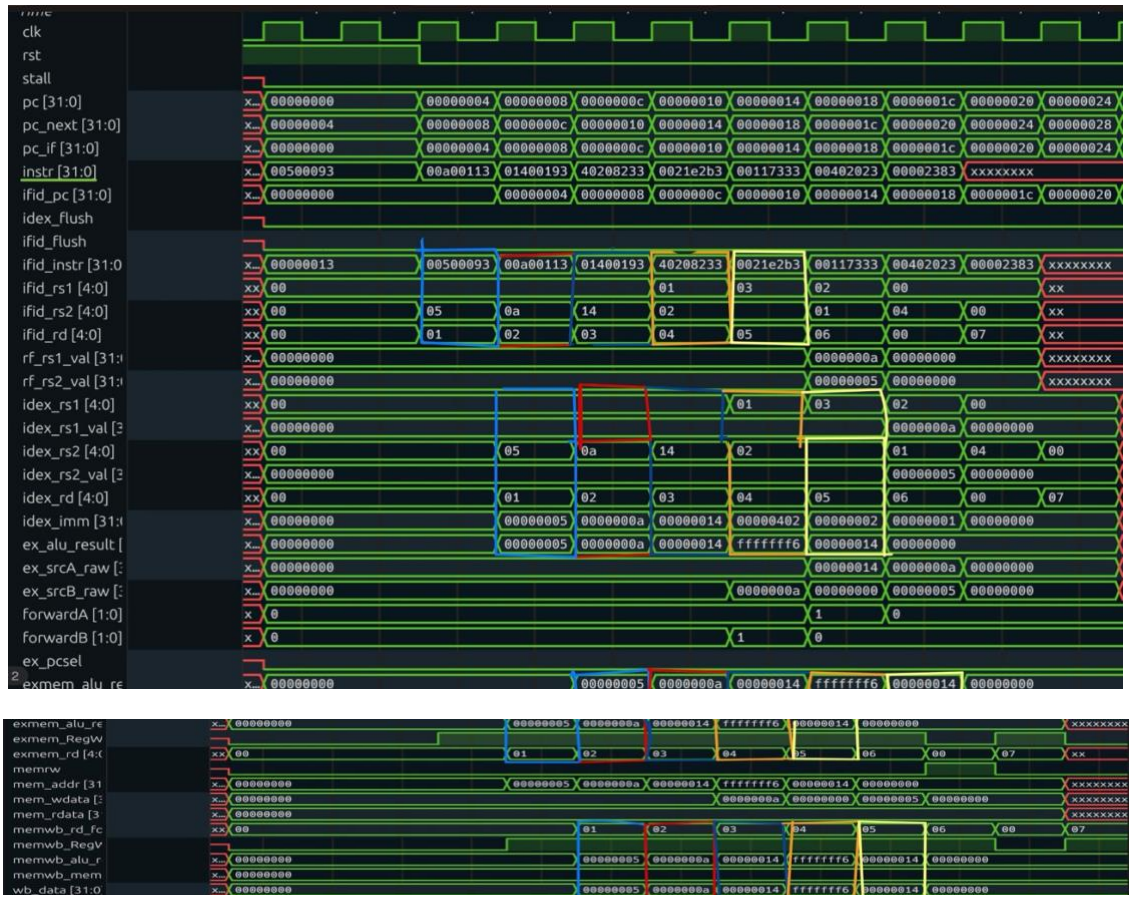

Figure 12 : Simulation without hazard

12.10.2 Hazard

This test verifies load-use hazard detection. A one-cycle stall is inserted when a dependent instruction follows a load.This is observed with stall = 1, PC hold, and a bubble inserted in the pipeline before execution resumes.

```
00500093   # addi x1, x0, 5
00102023   # sw   x1, 0(x0)
00002103   # lw   x2, 0(x0)
002081B3   # add  x3, x1, x2      stall required
0011F233   # and  x4, x3, x1
00000013   # nop
00000013   # nop
00000013   # nop
```

Figure 13 : Simulation with data hazard

12.10.3 forwarding test

This test verifies ALU forwarding for dependent instructions, avoiding stalls.Forwarding is confirmed by ForwardA and ForwardB, which select results from later pipeline stages instead of the register file.

```
00500093   # addi x1, x0, 5
00A00113   # addi x2, x0, 10
002081B3   # add  x3, x1, x2     forwarding
40118233   # sub  x4, x3, x1     forwarding
002262B3   # or   x5, x4, x2     forwarding
0012F333   # and  x6, x5, x1     forwarding
00131393   # slli x7, x6, 1      forwarding
00000013   # nop
```

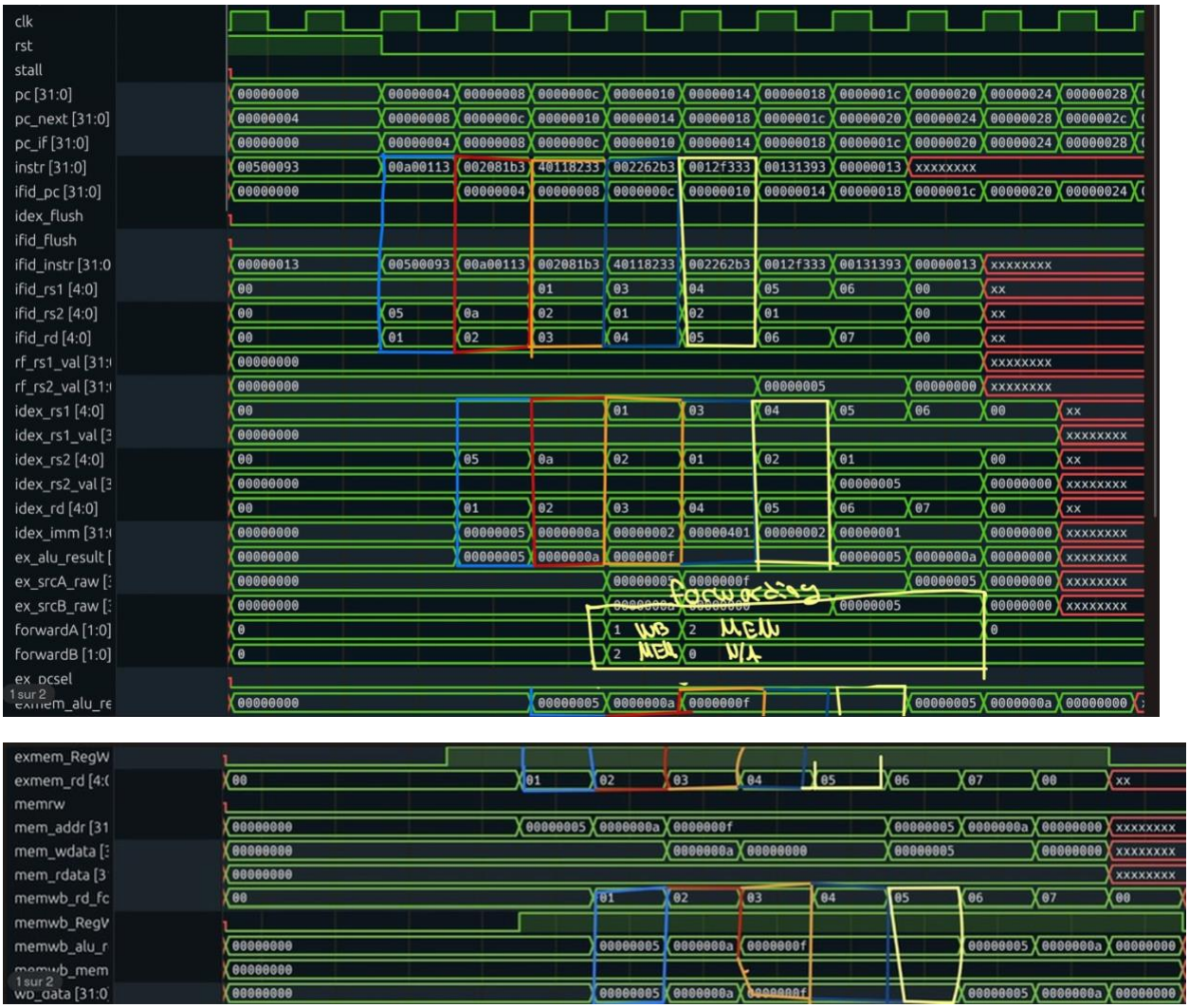

Figure 14: Simulation with forwarding

12.10.4 flush branch test

This test verifies control hazard handling. After a taken branch, wrong-path instructions are flushed.This is observed with flush = 1, where the IF/ID instruction is cleared and execution resumes at the branch target.

```
00500093   # addi x1, x0, 5
00500113   # addi x2, x0, 5
00208463   # beq  x1, x2, +8        branch taken
00100193   # addi x3, x0, 1        wrong-path instruction -> must be flushed
00200213   # addi x4, x0, 2         branch target instruction
004202B3   # add  x5, x4, x4
00000013   # nop
00000013   # nop
```

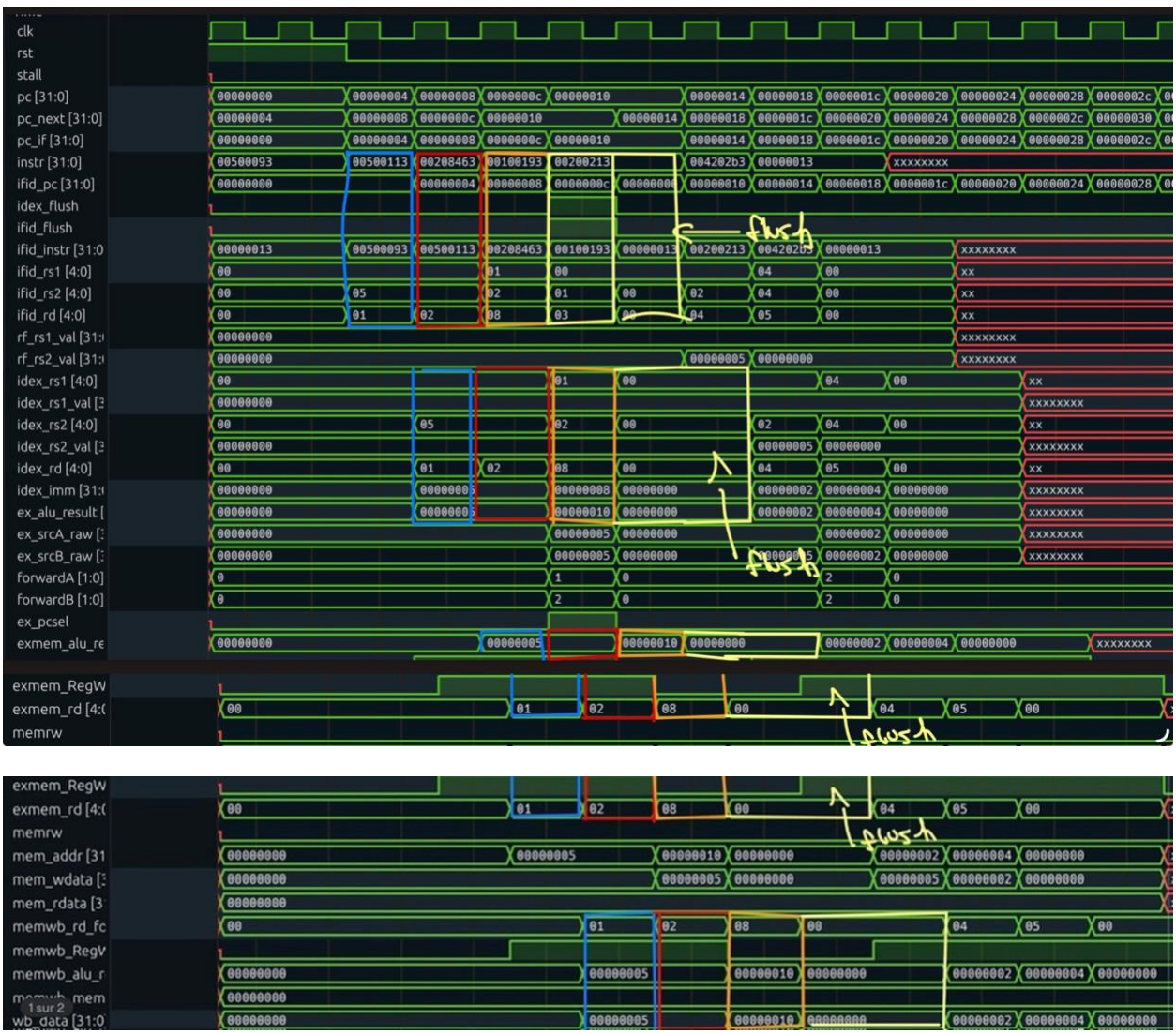

Figure 15 : Simulation with control hazard

13. Design analysis

13.1 Timing analysis

Timing analysis was performed using Yosys for synthesis and OpenSTA for static timing analysis.

Key results:

Metric	Result
Critical Path	R-R
Longest Delay	5.87 ns
Max clock Frequency	~ 170 MHz

These results confirm that the pipeline registers successfully isolate stage delays, allowing the processor to operate at a higher clock frequency compared to a single-cycle design.

13.2 Stall/ hazard perf analysis

Pipeline hazards were analyzed through simulation and architectural inspection. The processor implements both forwarding and hazard detection to reduce pipeline stalls.

Observed behavior:

Hazard Type	Resolution	Penalty
ALU Dependency	Forwarding	0 cycles
Load use Hazard	Stall + bubble	1 cycle
Branch Instruction	Flush	1 cycle

Forwarding removes most data hazards, while the hazard detection unit introduces stalls only when required.

13.3 Resource/area estimation

Hardware resource utilization was estimated using Yosys synthesis reports. The synthesis results provide an approximation of the processor's hardware footprint in terms of logic cells, registers, and memory elements.

Resource	Count	Descr
Flip-Flops	1550	Pipeline reg, reg file
Logic cells	6367	ALU, CU, FW, Hazard
Muxes	2573	--
Adders	2	ALU, PC
Memory Blocks	Excluded from count	Instruction and data memory

These metrics estimate the processor's hardware complexity based on synthesis results. The design was **not implemented on an FPGA**, so the reported values represent synthesis-based estimates rather than measured hardware utilization.

13.4 Transistors Count Estimation

The transistor count of the pipelined RISC-V processor was estimated by assigning an approximate CMOS transistor cost to each synthesized standard cell reported by Yosys. The synthesis results provide the number of instances of each cell type used in the design, which are then mapped to their transistor-level equivalents.

```
=== riscv_core_pipelined ===

        +----------Local Count, excluding submodules.
        |
     4755 wires
     7978 wire bits
      222 public wires
     3383 public wire bits
        9 ports
      166 port bits
     6367 cells
      926   $_ANDNOT_
       78   $_AND_
       11   $_DFF_P_
     2573   $_MUX_
       49   $_NAND_
       86   $_NOR_
      221   $_NOT_
      112   $_ORNOT_
      562   $_OR_
      992   $_SDFFCE_PN0P_
      121   $_SDFFE_PP0N_
        2   $_SDFFE_PP0P_
        3   $_SDFFE_PP1N_
      153   $_SDFF_PN0_
      277   $_SDFF_PP0_
       50   $_XNOR_
      138   $_XOR_
       13   $scopeinfo
```

Figure 16: Results from Yosys synthesis

Assumptions

Each standard cell is approximated using a typical CMOS implementation:

Cell type	Transistor per Cell
NOT	2
NAND/NOR	4
AND/OR/	6
XOR/NXOR	10
MUX (2:1)	12
Flip-Flop	30

Calculation Table

Category	Cells	Count
NOT	221	442
NAND/NOR	49+86 = 135	540
AND/OR/	78+562+926+112=1678	10 068
XOR/NXOR	138+50 = 188	1 880
MUX (2:1)	2573	30 876
Flip-Flop	1559	46 770

Total

$$442 + 540 + 10068 + 1880 + 30876 + 46770 = 90576$$

This estimate includes only the synthesized core logic and excludes memory blocks, which would significantly increase the total transistor count in a real implementation.

The high number of flip-flops and multiplexers reflects the pipelined design, where pipeline registers and forwarding/control logic add hardware complexity compared to a single-cycle processor.

Overall, this provides a reasonable gate-level approximation of the processor's complexity, though a full implementation would include additional overhead from memory and physical design.

13.5 Design Trade-offs

The pipelined architecture improves instruction throughput but increases design complexity compared to a single-cycle processor. Additional hardware such as pipeline registers, forwarding logic, and hazard detection units are required to maintain correct execution.

14.Performance Analysis

Processor performance is mainly determined by the pipeline depth and hazard handling mechanisms. The five-stage pipeline enables instructions to execute concurrently across stages, improving throughput compared to a single-cycle processor.

Since the processor was evaluated at the **RTL level and not implemented on FPGA**, the following metrics are derived from the architectural behavior of the pipeline rather than measured benchmark results.

Metric	Value
Pipeline depth	5 stages
Latency	5 cycles
Ideal CPI	~1
Max Throughput	1 instruction/cycle

Once the pipeline is filled, multiple instructions execute simultaneously across different stages, allowing the processor to sustain instruction completion close to **one instruction per cycle**, with occasional stalls caused by load-use and control hazards.

15. Power Analysis

15.1 Switching Activity Analysis

Switching activity was analyzed using a VCD file from simulation and evaluated in OpenSTA. The results are shown in Figure 12.

```
Group                   Internal  Switching    Leakage      Total
                           Power      Power      Power      Power (Watts)
----------------------------------------------------------------
Sequential             1.05e-03   3.25e-05   1.19e-04   1.20e-03  68.2%
Combinational          2.67e-04   9.45e-05   1.99e-04   5.60e-04  31.8%
Clock                  0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
Macro                  0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
Pad                    0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
----------------------------------------------------------------
Total                  1.31e-03   1.27e-04   3.17e-04   1.76e-03 100.0%
                          74.8%       7.2%      18.0%
```

Figure 17: Power Simulation

Most power is consumed by sequential elements (68.2%) due to frequent toggling of pipeline registers, while combinational logic contributes 31.8%. Internal power dominates, with smaller contributions from switching and leakage. Clock power is reported as zero due to the assumption of an ideal clock network.

15.2 Dynamic Power Estimation

Total Power = 1.76 mW

- Internal Power: 1.31 mW
- Switching Power: 0.127 mW
- Leakage Power: 0.317 mW

Dynamic power is primarily dominated by internal power in sequential elements, reflecting the register-intensive nature of the pipelined design. While pipelining improves timing performance, it introduces additional registers, which increases overall power consumption. The relatively low switching power suggests efficient signal activity, while leakage remains a noticeable component of total power.

The total estimated power consumption of 1.76 mW indicates an energy-efficient design at an operating frequency of approximately 170 MHz. This demonstrates a good balance between performance and power, although the results are based on pre-layout analysis with ideal clock assumptions.

16.Limitation of the design

Although the processor successfully implements a functional 5-stage pipelined RV32I architecture, several limitations remain in the current design. Pipeline hazards can introduce stalls that temporarily reduce performance. Load-use data hazards require the pipeline to stall for one cycle, even when forwarding is used. Additionally, control hazards caused by branch and jump instructions may require flushing instructions in the pipeline, which also reduces efficiency.

Another limitation is that the processor supports only the RV32I base instruction set and does not include extensions such as multiplication and division (RV32M), floating-point operations (RV32F), or compressed instructions (RVC).

17.Summary

This project presents the design and implementation of a 32-bit 5-stage pipelined RISC-V processor supporting a subset of the RV32I instruction set. The processor was implemented in Verilog HDL using a Harvard architecture with separate instruction and data memories. The design extends a previously developed single-cycle processor by introducing pipeline stages for instruction fetch, decode, execute, memory access, and write-back. Pipeline registers, forwarding logic, and hazard detection mechanisms were implemented to handle data and control hazards. Simulation results verify correct instruction execution and demonstrate the expected behavior of the pipelined architecture.

18.Future work

Several improvements could further enhance the processor design. One possible extension is the implementation of the complete RV32I instruction set and additional instruction set extensions such as RV32M for multiplication and division.

Performance could also be improved by introducing branch prediction mechanisms to reduce pipeline stalls caused by control hazards. Another important enhancement would be the addition of cache memory systems for both instruction and data memory, which would reduce memory access latency and improve overall system performance.